

My server said 1672, my CLI said 1585

Emil Lindfors | August 23, 2026

Some time in mid-August I asked the same question twice and got two answers. On the command line:

```
$ wren cube query --cube order_revenue --measures gross_revenue,net_revenue
gross_revenue  net_revenue
      1672.0      1585.0
```

Net revenue, 1585.00. In the chat UI, going through my own MCP server against the same DuckDB file, revenue was 1672.00. This is Jaffle Shop: 100 customers, 99 orders, one quarter of 2018. I wrote both paths.

The difference is whether returned orders count, and neither number is a SQL bug. The rule that picks one lives in `knowledge/rules/revenue.md`, which says revenue means net here. The CLI loads that file. My server read the schema and stopped there, so it answered gross and sounded exactly as confident as the CLI did.

What makes it worse is that `customers.customer_lifetime_value` also sums to 1672. Its own column description said “across all completed orders”, and it sums every status — it is `SUM(orders.amount)` with extra steps. So a model reaching for the obvious-looking revenue column gets gross, silently, and a model reaching for the rule gets net. Two fields, both called revenue in conversation, disagreeing by design.

That file is a **semantic layer**, or the beginning of one. The idea is to keep the definitions of your business terms in one version-controlled place (measures, dimensions, the join paths between tables, and the rules a schema has nowhere to put) so that everything downstream compiles its queries from there instead of carrying its own private idea of what revenue means. Business Objects patented the concept in 1991, and every BI tool since has shipped some version of it, usually called a model or a universe or a cube.

What’s new is the consumer. A human analyst arrives already knowing that returns don’t count and that April is a partial month; nobody writes that down because everyone senior enough to be asked already knows. An LLM knows the schema and nothing else, so the conventions have to exist as files or they don’t exist at all.

I spent August building a chat UI over one of these, and then arguing for the approach internally. The commercial half of that argument (which SKUs, what Wren Cloud costs, which customer gets which) isn’t interesting outside the company. The rest of it is.

The empty directories

The implementation I picked is **Wren**, from Canner. It bills itself as GenBI, generative BI: governed text-to-SQL over twenty-odd databases, where the model is a set of reviewable files rather than a paragraph in a prompt. The layer is called MDL, and `wren-core` underneath it is Rust on Apache DataFusion, so a question compiles against the model and then out into whichever dialect the target database speaks. I've been driving DuckDB with it locally, and the same model would point at BigQuery or Snowflake without the measures changing.

There are two Wrens, and it's worth knowing which one you're reading about. The one you may remember was a three-service Docker stack with its own chat UI, AGPL; the repos froze on 30 April 2026 and it's archived to `legacy/v1`. Wren today is `pip install wrenai`: a Python CLI and SDK, Apache-2.0 on the CLI and core, with an MCP server on the side. There's no UI in the open-source build, which is why it suits building on.

The project scaffolds `knowledge/` and `cubes/` empty. So a fresh checkout demonstrates the plumbing and nothing the semantic layer is for, and filling those two directories turned out to be most of the month.

The layer is five constructs, all YAML and markdown in git:

- **models** — a table plus everything the schema doesn't say. Column descriptions carry tagged facts: `[enum]` for what a code value means, `[unit]` for currency and whether `0` is a real zero, `[null]`, `[time]` for grain and actual coverage, and `[magic]` for real rows that look like placeholders (order 65 is `completed` with `amount = 0`).
- **relationships** — the join path, declared once, with cardinality.
- **views** — a join written once. A cube sits on exactly one base object, so without `customer_orders` no cube could slice revenue by anything about the buyer.
- **cubes** — named measures and dimensions.
- **knowledge** — rules, glossary, caveats, and curated NL→SQL pairs, as markdown.

There's also `.wren/memory`, a LanceDB index over the schema and the curated pairs. It's gitignored and rebuildable, so I don't count it as a construct.

The cube is the piece that does the enforcing:

```
name: order_revenue
base_object: orders

measures:
  - name: gross_revenue
    expression: SUM(amount)
  - name: net_revenue
    expression: SUM(CASE WHEN status IN ('returned', 'return_pending') THEN 0
ELSE amount END)
  - name: avg_order_value
    expression: SUM(amount) / NULLIF(COUNT(*), 0)
```

```

dimensions:
  - name: status
    expression: status

time_dimensions:
  - name: order_date
    expression: order_date

```

The return rule sits inside the expression, so everyone who asks for `net_revenue` gets it applied, and the grammar has no way to express an invalid join or an undefined aggregate. Three more things in Jaffle Shop that the schema can't carry, all of which caught me before I wrote them down:

- **Denominators.** 38 of 100 customers never ordered. Average orders is 1.60 over the 62 who did and 0.99 over everyone. Repeat rate is 46.8% over 62 and 29% over 100. The rules file picks one and says which.
- **Enum meanings.** `return_pending` vs `returned` — which one has already cost you the money?
- **The clock.** Orders stop on 2018-04-09, so “last month” returns an empty table. The caveats file says to translate relative dates to absolute ones.

That's 99 rows and one quarter. Spider 2.0's average database has 812 columns.

Why not just let it write SQL?

That question came up every time I showed this to anyone, and it's fair, because on the benchmarks everybody quotes the models are already good. Here's ol-preview across three of them, all three figures from the [Spider 2.0 paper](#):

Benchmark	What it is	Score
Spider 1.0	academic schemas	91.2%
BIRD	semi-real	73.0%
Spider 2.0	enterprise	21.3%

[Spider 2.0](#) is the one that looks like a customer. Its databases average 812 columns, often past a thousand; the gold SQL is frequently over 100 lines; the tasks are multi-dialect and multi-step. 632 of them. And the ceiling isn't moving quickly — the best score on the [BIRD leaderboard](#) is 81.95% from December 2025, against a 92.96% human baseline, and nothing has beaten it in the eight months since.

One caveat should attach itself to every number in this section, including the 96% ones somebody will quote at you. A [CIDR 2026 paper](#) re-checked the labels and found annotation error rates of 52.8% in BIRD Mini-Dev and 66.1% in Spider 2.0-Snow. Correcting

them moved leaderboard rankings by up to three positions across the five methods they re-scored, with CHESS going from 4th to 1st.

What closes the gap in the studies I could find is the layer rather than the model. [Sequeda et al.](#) (GRADES-NDA 2024) put GPT-4 on an enterprise insurance schema and went from 16% against the raw schema to 54% against a knowledge graph. [dbt Labs](#) (April 2026) took Claude Sonnet 4.6 from 62.5% to 100% on in-scope questions with a semantic layer under it. [Anthropic](#) (June 2026) reported 21% to 95% of business queries automated in their internal analytics, which they attribute to “skills” — the semantic layer plus canonical datasets, lineage, a query corpus and business context. Not the layer on its own, and worth repeating when someone cites the number at you.

The finding I keep coming back to isn’t in any of those headlines. In the dbt study, adding three more models took the layer to 98–100% in scope and lifted raw text-to-SQL to 84–90%, depending on which model you asked. Through the layer, which LLM you used barely mattered. The modelling did more work than the model choice, which is either encouraging or depressing depending on whether you sell modelling.

In that same dbt study the semantic layer scored 0% on out-of-scope questions, where raw text-to-SQL scored 70–100%. That reads as a loss until you look at how each one fails. Join orders to line items before aggregating and a three-item order triples its revenue. The query runs. It returns a believable figure, off by about 30%, with a tidy summary paragraph explaining what it means. Nothing in the result indicates that anything happened.

The semantic layer’s version of failing is `I can't answer that - churn_reason isn't in the model.` You extend the model. You don’t spend a week debugging a number nobody caught.

For a user group that can read the generated SQL, silent fan-out is an annoyance. For the group I want to put this in front of, the ones who have been waiting three weeks for a dashboard, it’s disqualifying.

The stack

```
browser → chat-ui → mcp-wren-charts → wren CLI (MDL) → DuckDB
          (LLM)      (MCP tools)      semantic layer
```

Four containers, one `docker compose up`. The Next.js service holds the API key, so the browser never sees it, and the MCP server isn’t published to the host at all. The model has no database credentials and no SQL escape hatch: everything it can do is a tool the MCP server exposes. Because it has no privileged access it’s swappable behind one env var — Azure, OpenAI, Anthropic, OpenRouter, or a local vLLM or Ollama endpoint. It does need to be competent at multi-step tool use, since one answer is usually four or five calls.

Twelve tools: three for context (`get_instructions`, `get_knowledge`, `recall_context`), six for models and cubes (`list_models`, `describe_model`, `list_cubes`, `query_cube`, `run_sql`, `suggest_questions`), and three for charts (`query_and_chart`, `get_chart`, `compose_dashboard`). The chart tools are the part Wren's own MCP server doesn't have, and they were the reason to write my own.

Charts come back as MCP Apps `ui://` resources rather than images. `assistant-ui` mounts the widget in a sandboxed iframe and bridges its JSON-RPC calls back through an API route, so chart type, x and y stay switchable in place without re-running the query. Threads and messages live in IndexedDB, and a restored chart widget comes back live after a reload because the rows, the spec and the resource are all inside the persisted tool result.

Two datasets run in the same UI, with a switcher in the header and separate threads per dataset. Jaffle Shop over Wren, and about 1,800 Norwegian aquaculture sites over a `strata` lakehouse, which has weekly lice counts back to 2012 and is the one where the time dimensions and period-over-period comparison actually get exercised. `strata` calls a cube a metrics view; the MCP server translates at the boundary so both halves of the demo read the same.

The parts that fought back

Four things went wrong along the way, in roughly this order.

`wren memory list|check|export|dump` all died with an `ImportError`. Those four commands read the LanceDB table through the `lance` Python bindings, which `wrenai[memory]` doesn't declare, so the extra installs and the commands still don't run. Adding `pylance` to the requirements fixed it.

Charts rendered as static PNGs in the chat UI while rendering as interactive widgets in Claude Desktop. SEP-1865 has you declare `_meta.ui.resourceUri` on the tool, which is what Claude Desktop reads. `assistant-ui`'s AI SDK converter checks `output._meta.ui` and the flat `output._meta["ui/resourceUri"]` on the result, and never looks at the tool declaration. Both are defensible readings of the spec. Repeating the resource in both places is additive, so it works everywhere and nothing gets confused.

Then capability detection stopped working under compose. The Streamable HTTP transport with `sessionIdGenerator: undefined` builds a new server instance per request, so the client capabilities from `initialize` aren't available at `tools/call` time. Auto-detection only ever works over stdio. The compose file sets `WREN_FORCE_UI=1` and stops guessing.

And I had put thread history in `localStorage`, which was fine until two threads came to 26 KB, because every tool result embeds its full row set. The ~5 MB cap would have bitten early, so history moved to IndexedDB.

The stale build

On 18 August I pointed a fresh Claude session at the server, told it to go, and gave it no help. Six `query_and_chart` calls covering all five chart kinds, five `run_sql` calls including UNION ALL unpivots and scalar subqueries, two recalls. Every query succeeded on the first attempt, and the reconciliation checks it ran on its own confirmed the documented invariants to the cent.

The write-up it produced was more useful than the success rate, and it led with three problems: the cubes are documentation rather than an interface, recall is read-only, and the knowledge files that the column descriptions cite can't be reached from any tool. I read that, agreed with all three, and started planning the work.

All three had shipped hours earlier. The host was talking to a server process started before the commit, and I'd never restarted it. So I spent a while planning features I had already written, which is a stupid way to lose the time and an easy one. That's what `server_info` is for now: version, the tool list this build registers, and the MDL build time, so a session can tell what it's actually connected to. Running a field test without one was the real mistake.

The feedback that survived the restart was smaller and mostly cheap. Results now carry `row_count`, `truncated` and `elapsed_ms`; a failed query comes back with the engine's message plus a hint naming the tool that would have prevented it, so the agent corrects instead of retrying blind; recall returns JSON with distance scores and repo-relative paths instead of a fixed-width table full of Windows paths. The charting notes took longer and were the most useful thing in the report: one y column per chart became several, with stacked, grouped and 100%-normalized modes, a line-over-bars overlay on a second axis, currency and date formatting, sort and top-N, and deterministic jitter so 62 scatter points stop collapsing onto four vertical lines.

The one idea from it I still haven't built is letting a session mark a recalled NL→SQL pair as stale or wrong. Recall can be written to and can't be corrected, which is the wrong way round for a knowledge base that grows by accretion.

Where this leaves me

Share of employees actually using BI tools: 19% in 1998, 22% in 2009, 35% in 2019, 25% in 2022. Four figures from three firms with three methodologies, so it isn't a clean series and definitely isn't a decline. What it is, is flat, across twenty-five years in which both spend and capability went up a great deal. Seats are still priced per person: Power BI Pro at \$14/user/month, Tableau Cloud Standard from \$15, a Tableau Creator seat at \$75. The AI tiers land above the tiers people already don't use.

The queue is the structural reason. A business question ("is the Bergen site actually worse?") becomes a ticket, an analyst writes SQL, a dashboard ships, and the follow-up question goes back to the top. At Deputy, a VP of Customer Success asked for a dashboard and was told it would take nine months, because that's how long it would take to build;

every stakeholder group they had was unhappy with the data team. At [incident.io](#), custom SQL fell from 70% of queries to 5% after they moved metric definitions into a shared model in Omni. No chatbot involved in that one.

What I think is changing is who gets to ask the second question. If a CEO can ask “is the Bergen site worse” and then ask the four follow-ups without filing a ticket for each, the analytics function shifts from producing charts to maintaining the definitions those charts agree on. I’ve seen that land hardest in smaller companies, where the C-suite is close enough to the data to want to dig themselves and there was never an analytics team big enough to absorb the queue.

I’m less sure about the platform, and this is where a standard would help. Every vendor’s semantic layer is its own YAML dialect today, so the expensive part of the work, writing down what your company means by revenue, is locked to whoever you wrote it for. [Open Semantic Interchange](#) is the attempt to fix that: one vendor-neutral spec for metrics, dimensions and relationships, so a BI tool, a query engine and an agent can read the same definition. It was announced in September 2025 by Salesforce, Snowflake and dbt Labs among others, published a v0.1 spec on 27 January 2026, was accepted into the Apache Incubator on 10 July as [Apache Ossie](#) (renamed to stop colliding with every other OSI), and has grown from 17 organisations to over 50. Databricks and Dremio are in it too.

Wren’s MDL isn’t an Ossie format, and a move would be a conversion rather than a copy, so nobody should plan on models travelling between vendors unchanged today. The pieces are converging and they haven’t converged. Which means I don’t have a battle-tested concept to hand a customer, and I wouldn’t claim today’s shape is the one that’s still right in a year. It needs constant experimenting and a willingness to move when a better pattern shows up, and I’d rather say that than sell a roadmap.

Build or buy

Build versus buy resolves per customer, and for me it turns on whether there’s one project to maintain and share across several of them rather than a bespoke build each time. Databricks and Snowflake are good products. Lifting the analysis out of them and tailoring it to a specific company still seems worth doing, and I lean open source because I’m in this for the long haul with these customers and want them able to migrate when they outgrow a choice I made for them. That preference does a lot of work in what I end up recommending, so it’s worth being explicit that it’s a preference. I’m the AI capability lead at Fraktal Oslo, and all of the above is my opinion rather than a company position.

What I don’t have

A regression suite over the semantic model. There’s a `store_query` tool behind a `WREN_MCP_ALLOW_WRITE` flag that writes confirmed NL→SQL pairs back into `knowledge/sql/`, so

this week's good answer becomes next week's recall hit, and that's the intended counter-measure to the model going stale as the warehouse moves under it. Whether it works isn't something I can claim. Anthropic measured their own offline accuracy drifting from about 95% at launch to about 65% over a month before they treated it as an engineering problem. If I were putting this in front of a customer that suite would go in before the first deployment rather than after the first wrong number. I said as much in the presentation and then didn't build it, which is the usual order of these things.

The next thing I want to try is Power BI as the semantic layer via Microsoft's remote MCP server, which executes DAX and returns the full model schema. A Copilot licence is only needed for the `Generate Query` tool, so if our agent writes the DAX the Fabric Copilot capacity requirement goes away and the customer's existing model becomes layer two. It's still Preview, and row-level security is enforced for user auth but not for service-principal auth, which means anything headless bypasses RLS entirely. That has to be designed around before it can be delivered and I don't yet know how.

If you're running something like this in production, particularly the eval side, I'd like to hear how you're catching the drift. And if you've watched the analyst queue actually shorten at a company with more than a few hundred people, I'd like to hear that too, because so far I've only seen it work small.